



PROFESSIONAL JURY SUBMISSION REPORT

LeakShield Pro

DevSecOps Secret Detection, Risk Scoring & Public
Exposure Analysis System

Project Category: Cybersecurity / DevSecOps / Full-Stack Security Tool

Prepared For: Academic Jury Evaluation

Deployment: Vercel frontend and serverless scanner, Docker-ready backend stack

Core Purpose: Detect leaked credentials before attackers abuse them

1. Executive Summary

LeakShield Pro is a cybersecurity application designed to identify exposed secrets in source code, configuration files, deployment artifacts, public websites, and DevOps text. It combines a professional React dashboard with scanning logic, risk scoring, explanations, audit-friendly storage, caching, and deployment support.

What It Does

It scans pasted text, uploaded project folders, or public website assets and returns concrete findings such as leaked API keys, passwords, JWTs, database URLs, cloud keys, GitHub tokens, Stripe keys, Slack tokens, and private keys.

Why It Matters

Exposed secrets are one of the fastest ways for attackers to compromise accounts, databases, cloud infrastructure, payment systems, internal APIs, and CI/CD pipelines.

How It Helps

The tool does not only say “secret found.” It classifies the secret, calculates risk, shows the exact line/address, explains the attacker impact, and gives a remediation plan.

Professional Value

The project demonstrates secure engineering, full-stack architecture, risk modeling, deterministic security explanations, serverless deployment, and future AI extensibility.

Key Outcome

LeakShield Pro turns raw code or public assets into actionable security intelligence. This makes it suitable for pre-commit review, deployment checks, public website exposure checks, and academic demonstration of real-world DevSecOps principles.

2. Problem Statement and Objectives

Problem Statement

Modern applications often depend on API keys, cloud credentials, database passwords, tokens, private keys, and third-party service secrets. These values are sometimes accidentally committed to GitHub, copied into environment files, included in JavaScript bundles, printed in CI logs, or exposed through public configuration endpoints.

Once a credential becomes public, an attacker can use it directly or combine it with other exposed data to access private systems. The impact can include cloud billing abuse, database theft, account takeover, source-code compromise, fraudulent payments, phishing, and data loss.

Project Objectives

- Detect multiple classes of real-world leaked secrets.
- Support text, project folder, and public website scanning modes.
- Show exact evidence: file/address, line number, column, preview, and safe context.
- Hash and obfuscate sensitive values instead of storing plaintext secrets.
- Score risk from 0 to 100 and map it to LOW, MEDIUM, HIGH, or CRITICAL.
- Explain attacker impact, real-world consequences, and remediation steps.
- Provide a dashboard that a security analyst or developer can actually use.
- Remain extensible for future AI-based explanation enrichment.

Jury Defense Point

The project is not just a regex demo. It includes a complete workflow: acquisition, scanning, deduplication, risk modeling, remediation guidance, persistence, caching, browser history, public deployment, and documentation.

3. System Architecture

User / Developer -> React Dashboard -> API Adapter -> Vercel Serverless API or FastAPI Backend -> Scan Service / Scanner -> Detection Rules -> Risk Engine -> Explanation Engine -> Cache + Database / Browser History -> Structured Findings -> Dashboard Risk Report

Architecture Meaning

The project is separated into presentation, API communication, scanning logic, risk scoring, explanation generation, storage, and deployment layers. This separation improves maintainability because each layer has one clear responsibility.

Layer	Responsibility	Impact
React Dashboard	User input, scan mode selection, folder upload, website URL submission, visual results, history, filtering.	Turns complex security output into understandable evidence for users and jury evaluators.
API Layer	Receives scan requests and returns structured JSON responses.	Creates a clean boundary between UI and scanning logic.
Detection Engine	Applies rule patterns and extracts potential secrets.	Identifies dangerous data before deployment or after public exposure.
Risk Engine	Scores each finding based on severity, confidence, public context, and secret type.	Helps prioritize the most dangerous leaks first.
Explanation Engine	Converts technical findings into human-readable impact and remediation.	Makes the report useful for developers, security reviewers, and non-specialist judges.
Storage/Cache	Stores audit records and avoids duplicate repeated scanning.	Supports performance, review history, and repeatability.

4. What Each Important File Means

The table below explains the purpose of the major code files and how each one affects the system.

File	Meaning and Impact
frontend/src/App.jsx	Main React dashboard. It controls scan mode, user input, upload flow, history, result rendering, risk panels, and finding cards.
frontend/src/api.js	Browser API adapter. It sends scan requests, reads/writes local history, and falls back gracefully when the backend is unavailable.
api/scans.js	Vercel serverless scanning endpoint. It powers the deployed demo with text, folder, and public website scanning.
api/scans/[id].js	Vercel demo history endpoint. It explains that detailed history is stored in the browser for the free deployment.
backend/app/main.py	FastAPI application entrypoint. It initializes database/cache connections, configures CORS, exposes health checks, and mounts API routes.
backend/app/api/routes.py	REST API route layer. It exposes scan creation, scan listing, and scan detail loading while delegating business logic to ScanService.
backend/app/services/scan_service.py	Backend orchestration layer. It validates payload size, hashes content, checks cache, runs detection, scores risk, stores results, and returns response models.
backend/app/engines/detection/rules.py	Rule catalog. It defines secret signatures, severity, confidence, attacker impact, consequences, and remediation text.
backend/app/engines/detection/scanner.py	Detection engine. It executes rules, deduplicates matches, hashes and obfuscates values, calculates line/column positions, and extracts safe context.
backend/app/engines/risk.py	Risk engine. It converts findings into 0-100 scores and LOW/MEDIUM/HIGH/CRITICAL labels using severity, confidence, context, and secret type.
backend/app/engines/explanation.py	Explanation engine. It turns rule metadata and risk adjustments into clear security explanations.
backend/app/models.py	Database schema. It defines scans and findings tables with audit metadata, indexes, hashes, previews, and JSON explanations.
backend/app/database.py	Async SQLAlchemy setup. It creates the engine, session factory, dependency, and automatic table initialization.
backend/app/cache.py	Redis cache adapter. It stores and retrieves repeated scan responses as JSON to reduce duplicate work.
backend/app/schemas.py	Pydantic contracts. It defines request/response shapes for validation and stable API communication.

backend/app/config.py	Environment configuration. It centralizes database URL, Redis URL, CORS origins, cache TTL, and max scan size.
backend/app/optional_ai.py	Future extension point. It allows approved AI enrichment without making the core scanner dependent on external models.
docker-compose.yml	Local multi-service deployment. It runs PostgreSQL, Redis, backend, and frontend together for demonstration and testing.
vercel.json	Vercel deployment config. It builds the frontend and routes API/frontend paths for the hosted demo.

5. Backend Code Flow

FastAPI Application

`backend/app/main.py` creates the FastAPI app, initializes database and Redis connections during lifespan startup, applies CORS settings, exposes `/health`, and mounts the API router under `/api`. This gives the backend a clean lifecycle and makes it production-friendly.

REST Routes

`backend/app/api/routes.py` exposes three main security operations: create a scan, list scan history, and load a previous scan by ID. The route file is intentionally thin; it validates HTTP-level behavior and delegates the real logic to `ScanService`.

Scan Service

`ScanService` is the orchestration layer. It checks payload size, hashes the content, builds a cache key, checks Redis, runs detection rules, scores findings, generates explanations, persists data, refreshes database models, caches the response, and returns a structured scan report.

```
POST /api/scans
-> validate request
-> hash content and metadata
-> check Redis cache
-> run DetectionEngine
-> score each finding with RiskEngine
-> explain each finding
-> save Scan + Finding records
-> cache response
-> return JSON to dashboard
```

Security Impact

The backend stores hashes and obfuscated previews, not raw plaintext secrets. This is important because a security scanner should not become a second place where secrets are leaked.

6. Detection Engine and Rule Catalog

The detection engine works by applying rule objects to the provided content. Each rule contains a regex pattern plus security metadata: ID, secret type, severity, confidence, description, attacker impact, consequence, and remediation.

Secret Type	Severity	Impact
AWS Access Key ID	HIGH	Cloud account identification. If paired with a secret key, attackers can access AWS APIs.
AWS Secret Access Key	CRITICAL	Direct cloud authentication. Can lead to infrastructure takeover, data theft, and billing abuse.
GitHub Token	CRITICAL	Repository and workflow access. Can expose source code or compromise CI/CD pipelines.
OpenAI API Key	HIGH	API quota and endpoint access. Can create unexpected billing or abusive automated usage.
Google API Key	MEDIUM	Google service access. Risk depends on restrictions such as referrer, IP, and API scope.
Stripe Secret Key	CRITICAL	Payment platform access. Can affect customer, charge, refund, or transaction operations.
Slack Token	HIGH	Workspace API access. Can expose messages, users, channels, and integrations.
SendGrid API Key	HIGH	Email sending access. Can enable spam, phishing, reputation damage, and billing abuse.
Generic API Key	MEDIUM	Service identity exposure. Impact depends on the provider and key permissions.
Password Assignment	HIGH	Hardcoded credential. Can allow login to databases, panels, APIs, or reused accounts.
Database URL	CRITICAL	Direct datastore credential. Can expose records, queues, caches, or broker contents.
Basic Auth URL	HIGH	URL-embedded username/password. Can expose protected endpoints or upstream services.
Bearer Token	HIGH	Replayable session/API token. Works until expiration or revocation.
JWT Token	HIGH	Application authorization token. Can impersonate a subject if still valid.
Private Key	CRITICAL	Cryptographic identity material. Can affect SSH, TLS, signing, or encrypted data trust.

Detection Process

1. Loop through every rule.
2. Find pattern matches in the content.
3. Extract only the actual secret value where possible.
4. Ignore example/dummy placeholders to reduce false positives.

5. Deduplicate findings by rule, value hash, location, and source.
6. Calculate line and column numbers.
7. Generate a safe value preview and context snippet.

This design makes the scanner explainable and extendable: adding a new provider key usually means adding one new rule without changing the whole application.

7. Risk Scoring Model

The scoring system converts a detected secret into a practical risk number. The goal is to prioritize what should be fixed first.

Severity	Base Score	Meaning
LOW	20	Weak or contextual signal that may still require review.
MEDIUM	45	Potential credential or service identity exposure.
HIGH	70	Likely exploitable credential or token.
CRITICAL	88	Direct authentication material or high-impact platform secret.

Adjustments

- **Confidence multiplier:** Rules with higher confidence influence the score more strongly.
- **High-risk context:** Terms such as production, live, public repo, GitHub, exposed, website, or client increase risk.
- **Low-risk context:** Terms such as test, dev, staging, sandbox, local, or mock reduce risk.
- **Direct-authentication secrets:** Private keys, database URLs, AWS secret keys, and Stripe secret keys receive extra risk.
- **Public address:** If a finding appears in a public website asset or sensitive public path, the score increases.

Risk Levels

Range	Label	Action
0-34	LOW	Review and monitor.
35-64	MEDIUM	Investigate and rotate if confirmed.
65-84	HIGH	Rotate quickly and remove exposure.
85-100	CRITICAL	Immediate revocation, incident review, and redeployment.

8. Frontend Dashboard Explanation

The React dashboard is the analyst console. It presents three acquisition modes: raw text, project folder upload, and public website scanning. It also visualizes risk, displays finding cards, lets the user filter findings, and stores recent scan history in browser local storage.

Key UI Functions

- **Text mode:** Useful for environment files, logs, copied code, and quick demos.
- **Folder mode:** Reads multiple local text files, skips binary/media formats, and sends a safe structured payload for scanning.
- **Website mode:** Accepts a public URL and asks the serverless API to fetch HTML and same-origin assets.
- **History:** Saves scan summaries and details in local storage for the deployed Vercel demo.
- **Finding cards:** Show rule ID, secret type, risk badge, score, address, line/column, impact, and remediation.

Impact on Jury Presentation

The UI demonstrates that the project is usable, not only technically correct. A reviewer can paste a leak, press one button, and immediately see prioritized cybersecurity evidence.

9. Public Deployment and Serverless Scanner

The deployed demo uses `api/scans.js` as a Vercel serverless endpoint. This makes the website functional on Vercel without requiring the full Docker Compose backend stack during a jury demo.

Serverless Scanner Capabilities

- Supports text, folder, and website scan modes.
- Includes expanded provider-specific rules such as GitHub, OpenAI, Google, Stripe, Slack, and SendGrid.
- Skips large binaries and ignored folders like `node_modules`, `.git`, `dist`, and `coverage`.
- Fetches public website assets with timeout and concurrency limits.
- Scans common risky public paths such as `/.env`, `/config.json`, `/api/env`, and source maps.
- Returns recommendations, public exposure counts, scanned addresses, and skipped addresses.

Why This Matters

This design makes the online demo self-contained. The jury can open the public link and test scanning behavior immediately, while the backend folder still demonstrates a more complete production architecture with FastAPI, PostgreSQL, and Redis.

10. Database, Cache, and Privacy Design

Database Tables

Table	Purpose	Important Fields
scans	Stores one scan summary.	id, source_name, content_hash, overall_score, overall_level, finding_count, metadata, created_at
findings	Stores each detected issue.	rule_id, secret_type, severity, risk_score, value_hash, value_preview, line/column, context_snippet, explanation

Privacy Measures

- Secret values are hashed with SHA-256.
- Only obfuscated previews are shown, such as `abcd...wxyz`.
- Context snippets are escaped before returning to the UI.
- Cache keys are based on content/metadata hashes to avoid storing raw content in the key.
- Browser demo history is local to the user’s browser.

Redis Cache Impact

Redis avoids repeated work by returning a cached scan when the same content and metadata are submitted again. This improves response time and reduces duplicate database writes.

11. Security Impact of the Project

Cloud Security

AWS keys, cloud tokens, and private keys can lead to unauthorized infrastructure access. LeakShield highlights these as high or critical risk.

Application Security

JWTs, bearer tokens, and API keys can let attackers impersonate users or services. The tool shows replay risk and remediation guidance.

Database Security

Database URLs can expose full data stores. LeakShield treats them as critical because they contain both network location and credentials.

CI/CD Security

GitHub tokens and source-code leaks can compromise pipelines and supply chains. The scanner treats repository and workflow tokens as critical.

Public Website Security

Website scanning checks if secrets are visible in public HTML, JavaScript bundles, source maps, or public config endpoints.

Operational Response

Findings include concrete remediation: revoke, rotate, remove from public files/history, redeploy, and review provider logs.

Real-World Example

If a production database URL appears in a public JavaScript bundle, an attacker may obtain the username, password, hostname, and database name. LeakShield flags it as critical, shows the exposed address, recommends rotating database credentials, restricting network access, removing it from public assets, and checking logs for suspicious activity.

12. Testing, Limitations, and Future Scope

Testing

The backend includes a unit test for production password leak detection and risk scoring. The frontend build is validated through Vite, and the deployed Vercel path can be tested by submitting sample text or public URLs.

Known Limitations

- Regex-based detection can produce false positives when random strings resemble secrets.
- Highly unusual secret formats may be missed until new rules are added.
- Website scanning is limited by public access, response type, timeout, and same-origin asset discovery.
- The free Vercel demo stores scan history in the browser rather than a shared production database.
- Actual incident response still requires provider-side log review and credential revocation.

Future Scope

- Add more provider-specific rules for Azure, GCP service accounts, Firebase, Supabase, Twilio, Mailgun, and OAuth secrets.
- Add GitHub pull request scanning and CI/CD release blocking.
- Add role-based user accounts and team scan history.
- Add PDF export from scan results.
- Add optional model API enrichment after redacting actual secret values.

13. Optional AI/API Integration Recommendation

The project already works without AI. This is a strength because the core scanner remains deterministic, explainable, fast, and privacy-aware. However, an external model API can be valuable as an optional enhancement.

Recommended Design

```
Detection Engine finds leak
-> Secret value is masked/redacted
-> Safe context and metadata are sent to model API
-> Model improves explanation, summary, and remediation
-> Core risk score still comes from deterministic engine
```

Why Not Replace the Current Engine?

- Rules are faster and cheaper for exact secret signatures.
- Rules are explainable and easier to defend in front of a jury.
- Sending raw secrets to a third-party model can create privacy risk.
- AI is best used for explanation and prioritization after detection, not as the only detector.

Professional Recommendation

Keep the current scanner as the trusted foundation. Add a model API only as a redacted AI analysis layer for richer explanations, report writing, and remediation planning.

14. Jury Defense Summary

What Makes This Project Strong

- It solves a real cybersecurity problem: accidental secret leakage.
- It includes both local/full-stack architecture and hosted Vercel demo architecture.
- It is modular, making new secret rules easy to add.
- It considers privacy by hashing and obfuscating secret values.
- It explains attacker impact and remediation, making it practical for real users.
- It demonstrates frontend, backend, API design, security engineering, deployment, and documentation skills.

Short Viva Answer

LeakShield Pro detects leaked secrets in code, folders, and public websites. It uses rule-based detection, risk scoring, safe secret handling, and human-readable remediation guidance to help developers remove exposed credentials before attackers exploit them.

Conclusion

LeakShield Pro is a professional DevSecOps security tool that demonstrates practical engineering and cybersecurity awareness. It identifies exposed credentials, explains their real-world impact, prioritizes the highest-risk findings, and gives clear remediation steps. The project is suitable for academic evaluation because it is functional, defensible, extensible, and aligned with real-world secure software development practices.